

Artificial Intelligence Basics Course Summary

Generated by Scribe.AI

December 9, 2024

1 Key Terms

1.1 Supervised Learning

- **ImageNet**: A large dataset of images used for training and evaluating image recognition models. It contains millions of images across thousands of categories.
- **MNIST**: A large database of handwritten digits used for training and testing in machine learning, particularly image recognition.
- **CIFAR-10**: A large dataset of 32x32 color images, commonly used for training and evaluating image classification models.
- **Training Data**: The subset of a dataset used to train a machine learning model.
- **Held Out Data**: A subset of the training data used for hyperparameter tuning and model selection.
- **Test Data**: A subset of the dataset used to evaluate the performance of a trained machine learning model on unseen data.
- **Generalization**: The ability of a machine learning model to perform well on unseen data.
- **Underfitting**: A phenomenon where a model is too simple to capture the underlying patterns in the data, resulting in poor performance on both training and unseen data.
- **Bias**: The error introduced by approximating a real-world problem with a simplified model.
- **Variance**: The error introduced by the model's sensitivity to small fluctuations in the training data.
- **Hyperparameters**: Parameters that control the learning process of a machine learning model, but are not learned from the data itself.
- **Regularization Coefficient (λ)**: A hyperparameter that controls the strength of the regularization penalty.
- **Learning Curves**: Graphs that show the training and testing performance of a model as a function of the training set size or model complexity.
- **S-fold Cross Validation**: A model evaluation technique that divides the data into S folds, using each fold as a validation set once while training on the remaining folds.
- **Data Analysis Pipeline**: A sequence of steps involved in analyzing data, including data collection, model design, parameter inference, and model application.

1.2 Unsupervised Learning

- **Matrix Factorization**: A technique to decompose a large matrix into the product of two smaller matrices, revealing latent factors that capture underlying relationships in the data.
- **Dimensionality Reduction**: Techniques that transform data from a higher-dimensional space to a lower-dimensional space, reducing the number of attributes while preserving important information.
- **Low Rank Matrix Approximations**: Approximating a high-dimensional matrix with a lower-rank matrix, effectively reducing the dimensionality of the data while preserving essential information.
- **Eigenvectors**: Vectors that, when multiplied by a matrix, only change in scale (not direction). In PCA, they represent the principal components.
- **Eigenvalues**: The scaling factors associated with eigenvectors. In PCA, they represent the variance explained by each principal component.

- **Embeddings:** A low-dimensional representation of data points, often used to project high-dimensional data into a lower-dimensional space for visualization or analysis.
- **Collaborative Filtering:** A recommender system technique that predicts user preferences based on the ratings of similar users.
- **Recommender Systems:** Systems that predict user preferences for items, such as movies or products, based on past behavior and similar users' preferences.
- **Clustering:** The process of grouping a set of objects into classes of similar objects. Objects within a cluster should be similar, and objects from different clusters should be dissimilar.
- **K-means clustering:** A commonly used clustering algorithm that partitions objects into K disjoint subsets, assuming the objects are p -dimensional vectors and using a distance measure (typically Euclidean distance) between them.
- **Euclidean distance:** The most commonly used distance measure for continuous data, defined as

$$d_E(x, y) = \sqrt{\sum_{i=1}^p (x_i - y_i)^2}, \text{ where } p \text{ is the number of attributes/dimensions.}$$

- **Hamming distance:** A distance measure used for bit/binary vectors that counts the number of bits that differ. It can be generalized to discrete data to count mismatches.
- **Matching coefficient:** A normalized Hamming distance, obtained by dividing the Hamming distance by the number of attributes/bits.
- **Inertia:** In the context of K-means, the sum of squared distances from each point to its cluster centroid. Minimizing inertia is the goal of K-means optimization.
- **Purity:** A measure of the extent to which clusters contain objects of a particular class. Calculated as
$$\text{purity}(C, G) = \frac{1}{N} \sum_{i=1}^K \max_j N_j \in G_i,$$
 where N is the total number of points, K is the number of clusters, and N_j is the number of examples of class j in cluster i .

1.3 Neural Networks

- **Perceptron:** A fundamental unit in neural networks, consisting of weighted inputs, a summation node, and an activation function that produces an output.
- **AlexNet:** One of the first deep convolutional neural networks to achieve high accuracy in image recognition competitions, notably the ImageNet challenge. It was a significant milestone in the development of deep learning.
- **Dense Neural Network:** A type of artificial neural network where each neuron in one layer is connected to every neuron in the next layer. This creates a fully connected network.
- **ConvNet:** Short for Convolutional Neural Network.
- **Artificial Neuron:** A computational model inspired by biological neurons. It takes weighted inputs, sums them, and applies an activation function to produce an output.
- **Convolution Layer:** A layer in a convolutional neural network that applies a filter (kernel) to the input data to extract features. The filter slides across the input, performing element-wise multiplication and summation at each position.
- **Pooling Layer:** A layer in a convolutional neural network that reduces the spatial dimensions of the feature maps by applying a pooling function, such as max pooling or average pooling.
- **Max Pooling:** A type of pooling operation where the maximum value within a defined region (e.g., 2x2) is selected as the output.
- **Average Pooling:** A type of pooling operation where the average value within a defined region (e.g., 2x2) is selected as the output.
- **Local Receptive Field:** The small region of the input data that a neuron in a convolutional layer is connected to. This leads to sparse connectivity.
- **Sparse Connectivity:** A characteristic of convolutional neural networks where each neuron is connected to only a few neurons in the previous layer, unlike fully connected networks.
- **Parameter Sharing:** A key characteristic of convolutional layers where the same set of weights (filter) is applied across different spatial locations in the input.
- **ReLU:** Short for Rectified Linear Unit, a non-linear activation function defined as $f(x) = \max(0, x)$.

- **Back-propagation**: An algorithm used to train artificial neural networks by calculating the gradient of the loss function with respect to the network's weights and updating the weights accordingly.
- **LeNet-5**: A convolutional neural network architecture specifically designed for the MNIST dataset.

1.4 Computer Vision

- **Bag of Features**: A representation of an image as a collection of visual features (codewords) and their frequencies, used for object recognition.
- **Category Models**: Representations of different object categories in a model space, based on their extracted features.
- **Model Space**: A multi-dimensional space where each point represents a category model, allowing for object categorization based on feature vectors.
- **Edge Detection**: The process of identifying points in an image where there is a significant change in intensity.
- **Edges**: Curves in an image where brightness changes rapidly.
- **Corners/Junctions**: Points in an image where there are significant changes in intensity in multiple directions.
- **Intensity Profile**: A plot of intensity values along a horizontal scanline of an image.
- **Image Derivatives**: Approximations of the rate of change of intensity in an image, often used for edge detection.
- **Finite Difference**: A method for approximating the derivative of a function using discrete values.
- **Linear Filter**: A filter that performs a linear transformation on an image, often represented as a convolution operation.
- **Moving Average**: A smoothing filter that averages the values in a local neighborhood of pixels.
- **Image Gradient**: A vector that points in the direction of the most rapid increase in intensity, with its magnitude representing the rate of change.
- **Canny Edge Detector**: An algorithm for edge detection that uses multiple steps, including gradient calculation, non-maximum suppression, and hysteresis thresholding.
- **Vision Pyramids**: Hierarchical representations of an image at multiple resolutions, used for feature extraction at different scales.
- **Gaussian Pyramids**: A type of vision pyramid that uses Gaussian blurring and subsampling to create a hierarchy of image representations.
- **Convolution**: A mathematical operation that combines two functions to produce a third function, often used in image processing for filtering and feature extraction.
- **Harris Detector**: A corner detection algorithm that uses the second moment matrix to identify corners in an image.
- **Harris Features**: Points in an image identified as corners by the Harris detector.
- **Corner Detection**: The process of identifying points in an image where there are significant changes in intensity in multiple directions.
- **Sum of Squared Differences (SSD)**: A metric used in corner detection to measure the difference in pixel intensities between a window and its shifted version.
- **Derivative of Gaussian Filter**: A filter used for edge detection that is created by taking the derivative of a Gaussian function.
- **Anisotropic Gaussian**: A Gaussian function with different standard deviations along the x and y axes.
- **Isotropic Gaussian**: A Gaussian function with the same standard deviation in all directions.

1.5 Reinforcement Learning

- **Reinforcement Learning**: A type of machine learning where an agent learns to make decisions by interacting with an environment and receiving rewards or penalties.

1.6 Search Algorithms

- **Minimax Search**: A search algorithm for two-player zero-sum games that aims to find the best move

for the current player, assuming the opponent also plays optimally.

- **Alpha-Beta Pruning:** A technique to reduce the search space in Minimax search by avoiding the evaluation of branches that cannot affect the optimal decision.
- **Game Tree:** A tree-like structure representing all possible sequences of moves in a game, with leaf nodes representing the outcomes.
- **Recursion:** A process where a function calls itself, eventually terminating in a base case.
- **Base Case:** The simplest form of a problem in a recursive approach, providing a direct solution without further recursion.
- **Recursive Case:** In a recursive approach, the same problem/method/data structure, but smaller, closer to the base case.
- **Breadth First:** A tree traversal method that visits nodes level by level.
- **Depth First:** A tree traversal method that traverses down a branch of a tree.
- **Binary Search Tree:** A tree data structure where each node has a value, its left child has a smaller value, and its right child has a larger value.
- **Problem Solving as Search:** An approach to problem-solving that involves exploring possibilities to find a solution.
- **States:** Descriptions of the configuration of an environment in a search problem.
- **Actions:** Activities that move from one state to another in a search problem.
- **Search Algorithm:** Determines which actions should be tried in which order to find a solution in a search problem.
- **Successor Function:** A function that maps a state to its possible successor states.
- **Dynamic Programming:** An optimization technique for recursive algorithms that avoids redundant calculations by storing and reusing previously computed values.
- **Graph:** A data structure consisting of nodes and edges, where edges connect pairs of nodes.
- **Rational Agent:** An agent whose actions are expected to maximize goal achievement, given available information.

1.7 Data Analysis & Evaluation

- **Bias-Variance Tradeoff:** The balance between model complexity and prediction accuracy. High complexity can lead to overfitting (high variance), while low complexity can lead to underfitting (high bias).

1.8 Fairness and Security in AI

- **Fairness:** The property of an algorithm or system to treat all individuals or groups equitably, without discrimination.
- **Disparate Treatment:** A type of discrimination where a decision-making process explicitly uses protected attributes (e.g., race, gender) to make decisions.
- **Disparate Impact:** A type of discrimination where a decision-making process, even without explicitly using protected attributes, disproportionately harms or benefits certain groups.
- **Privacy:** The right of individuals to control their personal information and how it is used.
- **Security:** The protection of data from unauthorized access, use, disclosure, disruption, modification, or destruction.
- **Algorithmic Opacity:** The difficulty in understanding how an algorithm arrives at its decisions.
- **Proxies for Discrimination:** Seemingly neutral variables that correlate with protected attributes and can indirectly lead to discriminatory outcomes.
- **Interpretable Results:** The ability to understand and explain the reasoning behind an algorithm's decisions.
- **Group Fairness:** A fairness criterion that requires equalizing outcomes across different groups.
- **Individual Fairness:** A fairness criterion that requires similar individuals to be treated similarly.
- **Automatic Prediction System:** A system that uses algorithms to make predictions automatically.
- **Data:** A collection of facts, figures, or information used for analysis.
- **Model:** A mathematical representation of a system or process used to make predictions or decisions.
- **Population Samples:** A subset of the population used to train a machine learning model.

- **True Population:** The entire population that a machine learning model is intended to make predictions about.

2 Problem Types

2.1 Fundamental AI Concepts

- **Understanding the Nature of Intelligence:** This problem explores the definition and characteristics of intelligence, both human and artificial. It examines various perspectives on what constitutes intelligence and how it can be measured or simulated.
- **Defining Artificial Intelligence:** This problem involves formulating a precise definition of artificial intelligence, considering different perspectives and historical contexts. The challenge lies in capturing the essence of AI while accounting for its evolving nature and diverse applications.
- **Passing the Turing Test:** This problem focuses on the implications of passing the Turing Test, a benchmark for machine intelligence. It explores whether passing the test truly indicates machine intelligence or merely the ability to mimic human behavior.
- **Overcoming the Limitations of Search:** This problem addresses the computational challenges associated with exhaustive search in complex games like chess and Go. It explores the need for alternative approaches, such as machine learning, to overcome the limitations of brute-force search.
- **Bridging the Gap Between Different AI Domains:** This problem investigates the similarities and differences between seemingly disparate AI domains, such as speech synthesis and game playing. It explores the potential for transferring knowledge and techniques across different AI applications.
- **Understanding the AI Grand Challenges:** This problem explores the reasons behind selecting specific AI problems, such as autonomous driving, as “Grand Challenges.” It examines the factors that make these problems particularly significant and challenging in the field of AI.
- **Understanding the Architecture of Advanced AI Systems:** This problem explores the architecture and design of complex AI systems, such as AlphaGo, which combine deep learning and search techniques. It examines how these systems integrate different components to achieve high performance in challenging tasks.
- **Representing Recursion:** This problem focuses on understanding and implementing recursive algorithms and data structures, such as the Fibonacci sequence and tree traversal.
- **Problem Solving as Search:** This problem involves formulating problems as search problems, defining states, actions, and goal tests, and choosing appropriate search algorithms. Examples include mazes and the 8-puzzle.
- **Optimizing Recursive Algorithms:** This problem addresses the efficiency of recursive algorithms by identifying and eliminating redundant calculations, as demonstrated with the Fibonacci sequence.
- **Searching Graphs and Maps:** This problem extends search algorithms to graph-based structures, considering multiple paths to a solution and the computational complexity of searching large state spaces. Examples include pathfinding on a campus map and the traveling salesperson problem.
- **Defining Rational Agents:** This problem focuses on understanding the concept of rational agents in AI, where the goal is to design agents that maximize goal achievement given available information.
- **Improving Reasoning and Modeling Technology:** This problem focuses on developing better technology for reasoning and modeling, aiming to improve the efficiency and effectiveness of automated reasoning systems.
- **Addressing Exponential Complexity Growth:** This problem addresses the challenge of exponential complexity growth in various problem domains, particularly in propositional reasoning, and explores strategies to mitigate this challenge.
- **Hamiltonian Path Problem:** This problem involves determining whether a path exists in a directed graph that visits each vertex exactly once.

2.2 Game Playing AI

- **Understanding the Differences Between Game Types:** This problem explores the differences between various game types, such as chess and soccer, in the context of AI. It examines how these

differences affect the design and implementation of AI algorithms for playing these games.

- **Designing Effective Game-Playing Systems:** This problem investigates the fundamental components and strategies involved in designing AI systems capable of playing games effectively. It explores the interplay between knowledge representation, planning, and decision-making in game-playing algorithms.

2.3 AI Safety and Risks

- **Mitigating AI Safety Risks:** This problem examines the potential risks associated with AI systems, such as adversarial attacks and biases. It explores methods for improving AI safety and ensuring responsible development and deployment.
- **Addressing Ethical Concerns in AI:** This problem focuses on the ethical implications of AI systems, particularly concerning bias and fairness. It explores the need for responsible AI development and deployment that considers societal impact and avoids perpetuating harmful biases.
- **Data Bias and Fairness:** This problem focuses on the presence of bias and unfairness in data used to train AI models, leading to discriminatory outcomes. Examples include bias in loan applications, hiring processes, and criminal justice risk assessments.
- **Data Privacy and Anonymization:** This problem explores the challenges of protecting individual privacy when using personal data for AI applications, even when data is anonymized. Examples include re-identification of individuals from seemingly anonymized datasets and the inherent identifiability of certain data types like genome sequences.
- **Algorithmic Transparency and Explainability:** This problem addresses the difficulty of understanding how complex AI models arrive at their decisions, making it hard to trust or debug them. The lack of transparency can hinder efforts to identify and mitigate bias.
- **Defining and Measuring Fairness in AI:** This problem focuses on the lack of consensus on how to define and measure fairness in AI systems. Different notions of fairness (e.g., group fairness, individual fairness) may conflict, making it challenging to develop fair and equitable AI systems.
- **The Risk of Spurious Correlations:** This problem highlights the risk of AI models learning spurious correlations from data, leading to inaccurate or misleading conclusions. An example is a study showing that asthmatic patients had a lower risk of dying from pneumonia due to aggressive care, but this correlation doesn't imply causation.
- **Addressing Algorithmic Bias:** This problem focuses on developing methods to detect, mitigate, and remove bias from AI models. Strategies include constraining the algorithm to ignore protected attributes, carefully curating training data, and post-processing techniques to remove learned biases.

2.4 Computer Vision

- **Addressing the Challenges of Computer Vision:** This problem investigates the difficulties in developing computer vision systems that can reliably interpret and understand images. It highlights the gap between initial expectations and the ongoing challenges in this field.
- **Bag of Features Object Categorization:** This problem involves using Bag of Features to categorize objects by mapping their feature vectors into a model space.
- **Edge Detection:** This problem focuses on identifying edges in images, which are curves where brightness changes significantly.
- **Corner Detection:** This problem involves identifying corners in images, which are points where the image's intensity changes significantly in multiple directions.
- **Vision Pyramids:** This problem focuses on representing an image as a "pyramid" of progressively smaller and blurred versions to extract features at different scales.
- **Object Detection and Recognition using CNNs:** Convolutional Neural Networks are used to identify and locate objects within images and videos. Examples include facial landmark detection, car detection in highway scenes, and fruit recognition.
- **Improving ImageNet Classification Accuracy:** This problem focuses on improving the accuracy of image classification models over time, as demonstrated by the decreasing error rate in the ImageNet challenge. AlexNet is highlighted as a significant milestone.
- **Comparing Dense and Convolutional Neural Networks:** This problem involves understanding

the architectural differences between standard fully connected neural networks and convolutional neural networks (CNNs), particularly concerning their connectivity patterns and data processing in multiple dimensions.

- **Convolution and Pooling Operations in CNNs:** This problem focuses on understanding the mathematical operations of convolution and pooling layers in CNNs, including the application of convolution filters (like Sobel Gx) and pooling functions (like max pooling) to reduce spatial dimensions.
- **Explaining Sparse Connectivity in Convolutional Layers:** This problem involves understanding why convolutional layers use sparse connectivity instead of dense connectivity, focusing on the concepts of local receptive fields and parameter sharing.
- **Understanding Pooling Layers:** This problem involves understanding the function of pooling layers in CNNs, including max pooling and average pooling, and their role in downsampling the input volume.
- **Understanding a Simple CNN Structure:** This problem involves understanding the architecture of a simple CNN, including the sequence of convolutional, ReLU, pooling, and fully connected layers, and their roles in image classification.
- **Analyzing Learned Convolution Filters:** This problem involves analyzing the learned convolution filters from a CNN, observing the patterns and textures they have learned to detect.
- **Multidimensional Convolution:** This problem involves understanding how multidimensional convolution operates on images with multiple color channels (e.g., RGB).
- **LeNet-5 Architecture for MNIST:** This problem involves understanding the LeNet-5 architecture, its layers (convolutional, pooling, fully connected), and how it processes the MNIST dataset.
- **CIFAR-10 Dataset and State-of-the-Art Models:** This problem involves understanding the CIFAR-10 dataset, its characteristics (32x32 color images, 10 classes), and the state-of-the-art accuracy achieved by various deep learning models on this dataset.

2.5 Machine Learning Algorithms

- **Developing Robust and Generalizable Algorithms:** This problem focuses on the design of algorithms that can adapt to new situations and generalize their knowledge to unseen scenarios. It explores the challenges in creating algorithms that are not only effective in specific tasks but also robust and adaptable.
- **Understanding Perceptrons:** The perceptron, a fundamental building block of neural networks, is introduced. Its structure and function are explained using a diagram.
- **Calculus in AI:** The slide briefly mentions the importance of calculus, specifically derivatives, partial derivatives, and finding maxima/minima/gradients, in AI algorithms.
- **Calculating the Derivative of the Sigmoid Function:** The slide shows the step-by-step derivation of the derivative of the sigmoid function using calculus rules. This is important for training neural networks.
- **Binary Classification:** Classifying data points into two categories (+1 or -1) using a linear model and a loss function.
- **Understanding Artificial Neurons:** This problem involves understanding the function of an artificial neuron, including weighted input summation, optional activation functions (like sigmoid and ReLU), and their role in introducing non-linearity.
- **Understanding the ReLU Activation Function:** This problem involves understanding the rectified linear unit (ReLU) activation function, its mathematical definition ($f(x) = \max(0, x)$), and its graphical representation.
- **Backpropagation for Neural Network Training:** This problem involves understanding the backpropagation algorithm used to train neural networks by adjusting weights to minimize the difference between predicted and desired outputs.
- **Biological Inspiration for Artificial Neural Networks:** This problem involves understanding the biological inspiration behind artificial neural networks, drawing parallels between biological neurons and perceptrons.
- **Using the MNIST Dataset:** This problem involves understanding the MNIST dataset, its characteristics (handwritten digits, size normalization), and its use in training and testing machine learning models.
- **Generalization and Overfitting:** This problem focuses on the ability of a model to generalize to unseen

data, and the risk of overfitting to the training data. The concepts are illustrated using cartoon robots representing well-generalized, well-fitted, and overfitted models.

- **Training \ testing errors:** This problem involves analyzing the prediction error on training and testing datasets to understand the bias-variance tradeoff and diagnose overfitting or underfitting. A graph visually represents the relationship between model complexity and prediction error for both training and testing samples.
- **Training and Testing:** This problem addresses the process of splitting data into training, held-out, and test sets to evaluate a machine learning model's performance and generalization ability. Cartoon robots are used to illustrate the training, practice exam, and final exam phases.
- **Hyper-Parameter Tuning:** This problem involves finding the optimal hyperparameters for a machine learning model to improve its performance. A cartoon robot is used to represent the process of adjusting hyperparameters.
- **Control the Model Complexity:** This problem explores methods for controlling model complexity, such as using the degree of a polynomial or the magnitude of weights in a model. A table shows weights for polynomials of different degrees.
- **Tuning on Held-Out Validation Data:** This problem focuses on using a held-out validation set to tune hyperparameters and prevent overfitting. A graph shows how different hyperparameter values affect accuracy on training, validation, and test sets.
- **Learning curves illuminate likely causes of error:** This problem uses learning curves (graphs of accuracy vs. model complexity) to diagnose underfitting and overfitting. Graphs show the relationship between model complexity and accuracy on training and test sets.
- **S-fold cross validation:** This problem describes a method for evaluating model performance and selecting hyperparameters using cross-validation. A diagram illustrates the process of splitting data into training and validation sets for each fold.
- **Bias-Variance Tradeoff:** Balancing model complexity (number of parameters) and prediction accuracy (RMSE) in recommender systems.
- **Supervised vs. Unsupervised Learning:** Supervised learning uses paired inputs/outputs to learn a function, while unsupervised learning uses only inputs to learn a function to simplify the data. Examples of unsupervised learning include clustering and matrix factorization.

2.6 Data Analysis and Modeling

- **Representing Linear Transformations with Matrices:** Matrices are used to represent linear transformations, which map vectors from one space to another while preserving linear combinations. This is fundamental to many AI algorithms.
- **Matrix Arithmetic:** The slide covers basic matrix operations such as addition, subtraction, scalar multiplication, and multiplication, which are essential for many AI computations.
- **Solving Systems of Linear Equations:** Systems of linear equations can be represented and solved using matrices and their inverses. This is crucial for various AI applications.
- **Matrix Transposition:** Matrix transposition is a fundamental operation that involves swapping rows and columns of a matrix. It's used in various AI algorithms for data manipulation.
- **Parallelizing Matrix Multiplication:** Domain decomposition is presented as a method for parallelizing matrix multiplication, improving computational efficiency in AI.
- **Linear Regression:** Finding the line that best fits a set of data points, minimizing the sum of squared distances between the points and the line.
- **Multiple Linear Regression:** Extending linear regression to include multiple independent variables to predict a dependent variable.
- **Curve Fitting:** Fitting a curve to data points, potentially using polynomial functions of higher degrees. This can lead to overfitting if the model is too complex.
- **Fit a cubic function with a degree-9 polynomial:** This problem demonstrates overfitting by fitting a high-degree polynomial to data, resulting in a model that fits the training data well but generalizes poorly. Three graphs illustrate the effect of increasing polynomial degree on the fit.
- **Regularized Linear Regression:** This problem introduces a method to prevent overfitting in linear regression by adding a regularization term to the loss function. The mathematical formulation of the

regularized loss function is presented.

- **The Data Analysis Pipeline:** This problem outlines the stages of a data analysis pipeline, highlighting potential issues at each stage. A grid of diverse figures represents different data types and models.
- **Matrix Factorization:** Approximating a high-dimensional data matrix as a product of two lower-dimensional matrices to reveal latent factors.
- **Dimensionality Reduction:** Transforming data from a higher number of dimensions to a lower number of dimensions.
- **Low Rank Matrix Approximations:** Reducing the rank of a matrix to achieve dimensionality reduction.
- **Collaborative Filtering:** Predicting a user's rating for an item based on the ratings of similar users.
- **Recommender System Algorithms:** Inferring missing ratings in a user-item matrix based on the ratings of similar users.
- **Modeling Systematic Biases:** Accounting for biases in user ratings and movie ratings to improve the accuracy of recommender systems.
- **Clustering Data:** Grouping a set of objects into classes of similar objects, where objects within a cluster are similar and objects from different clusters are dissimilar. This is a common form of unsupervised learning.
- **Evaluating Clustering Results Subjectively:** Assessing the quality of clusters by visually inspecting a reordered proximity matrix (heatmap) for a clear block pattern, or by using PCA to reduce dimensionality and visualizing the clusters in a lower-dimensional space.
- **Handling Non-Ideal Data in K-means:** Addressing the limitations of K-means clustering when the data does not meet its assumptions (e.g., non-spherical clusters, unequal variances, unevenly sized clusters).

2.7 SAT Solvers and Applications

- **Model Checking using SAT:** Utilizing SAT solvers to verify properties of systems, such as safety and liveness, by encoding the system's behavior and properties into a CNF formula.
- **Classical Planning with SAT:** Employing SAT solvers to find optimal plans by encoding the planning problem's constraints and goals into a CNF formula.
- **Solving Combinatorial Design Problems with SAT:** Applying SAT solvers to find solutions to complex mathematical structures with desirable properties, such as Latin squares or Steiner systems.
- **Applying SAT to Diverse Domains:** Using SAT solvers to address sub-problems in various fields, including test pattern generation, scheduling, optimal control, protocol design, and multi-agent systems.
- **Reducing SAT to 3-SAT:** Transforming a general SAT problem into an equivalent 3-SAT problem, where each clause contains exactly three literals, by introducing new variables.
- **Encoding Problems into SAT:** This problem focuses on translating various problem instances from different domains (e.g., planning, verification, combinatorial design) into a SAT formula that can be solved by a SAT solver.
- **Graph Coloring:** This problem involves assigning colors to vertices in a graph such that no two adjacent vertices share the same color.

3 Algorithm Solutions

3.1 Linear Algebra

- **Matrix Addition:** $A + B = C$, where $c_{ij} = a_{ij} + b_{ij}$
- **Matrix Scalar Multiplication:** $k \cdot A = B$, where $b_{ij} = k \cdot a_{ij}$
- **Matrix Multiplication:** $C_{ij} = \sum_{k=1}^n a_{ik} b_{kj}$
- **Matrix Inverse:** $AA^{-1} = A^{-1}A = I$
- **Solving Linear Equations using Matrix Inverses:** $Ax = b \implies x = A^{-1}b$

3.2 Machine Learning Algorithms

- **Perceptron Algorithm:** Iteratively update the weight vector w by adding $y_n x_n$ if the prediction $\text{sign}(w^T x_n)$ is incorrect. If the data is linearly separable, this algorithm will find the separating hyperplane.
- **Logistic Regression:** Minimize the logistic loss function $F(w) = \sum_{i=1}^N \log(1 + e^{-y_i w^T x_i})$ using stochastic gradient descent. The update rule is $w \leftarrow w + \alpha(\sigma(-y_n w^T x_n) y_n x_n)$, where $\sigma(z) = \frac{1}{1 + e^{-z}}$ is the sigmoid function.
- **Nearest Neighbor Classifier:** Assign label of nearest training data point to each test data point
- **Regularized Linear Regression:** $E(w) = RSS(w) + \lambda R(w)$, where $R(w) = \|w\|_2^2 = w_1^2 + \dots + w_k^2$ or $R(w) = \|w\|_1 = |w_1| + \dots + |w_k|$.
- **S-fold Cross Validation:** Split training data into S equal parts; use each part as validation data, others as training data; choose hyperparameter with best average performance.
- **K-means Clustering:** The algorithm iteratively assigns data points to the nearest cluster centroid, then recomputes the centroids as the mean of the points in each cluster, repeating until convergence.
- **Purity:** $\text{purity}(C, G) = \frac{1}{N} \sum_{i=1}^K \max_j N_j \in G_i$, where N is the total number of data points, K is the number of clusters, and N_j is the number of points of class j in cluster i .

3.3 Artificial Neural Networks

- **Derivative of Sigmoid Function:** $\frac{d}{dx} \sigma(x) = \frac{e^{-x}}{(1 + e^{-x})^2}$
- **ReLU Activation Function:** $f(x) = \max(0, x)$
- **Convolution Operation:** Element-wise multiplication of a filter with a section of the input image, followed by summation of the results.
- **Max Pooling:** Selecting the maximum value within a defined region (e.g., 2x2) of the input.
- **Average Pooling:** Calculating the average value within a defined region (e.g., 2x2) of the input.
- **Backpropagation:** An algorithm that calculates the gradient of the loss function with respect to the network's weights and uses this gradient to update the weights, iteratively minimizing the loss.

3.4 Image Processing

- **Image Sharpening:** $f_{\text{sharp}} = f + \alpha(f f_{\text{blur}})$
- **Finite Difference:** $\frac{df}{dx}[x, y] \approx F[x + 1, y] - F[x, y]$
- **Image Gradient Magnitude:** $\|\nabla f\| = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}$
- **Image Gradient Direction:** $\theta = \tan^{-1}\left(\frac{\partial f}{\partial x} / \frac{\partial f}{\partial y}\right)$
- **Anisotropic Gaussian:** $G_{\sigma_x, \sigma_y}(x, y) = \frac{1}{2\pi\sigma_x\sigma_y} \cdot e^{-\frac{1}{2}\left(\left(\frac{x^2}{\sigma_x^2}\right) + \left(\frac{y^2}{\sigma_y^2}\right)\right)}$
- **Isotropic Gaussian:** $G_{\sigma}(x, y) = \frac{1}{2\pi\sigma^2} \cdot e^{-\frac{r^2}{2\sigma^2}}$

3.5 Search Algorithms

- **Minimax Search:** A search algorithm for two-player zero-sum games that aims to find the best move for the current player, assuming the opponent also plays optimally. It uses an evaluation function to assign scores to game states and recursively explores the game tree, alternating between maximizing the score for the current player and minimizing the score for the opponent.

- **Alpha-Beta Pruning:** A technique to reduce the search space in Minimax search by avoiding the evaluation of branches that cannot affect the optimal decision. It uses alpha (best value for the maximizing player) and beta (best value for the minimizing player) values to prune branches.
- **Searching Graphs and Maps:** Finding paths in graphs and maps using search algorithms. States are nodes, actions are edges, and the goal is to find a path from the start node to the goal node.
- **8-Puzzle as a Graph:** Solving the 8-puzzle by representing it as a graph. States are tile configurations, actions are tile movements, and the goal is to reach the sorted configuration. Avoid exploring the same state twice.
- **Simple Search Abstraction Assumptions:** Static (world doesn't change), Discrete (finite states), Observable (states are known), Deterministic (actions have certain outcomes).
- **Pac-Man as an Agent:** Modeling Pac-Man as an agent interacting with an environment. The agent receives percepts from sensors, performs actions through actuators, and aims to achieve a goal (e.g., eating all dots).

3.6 Data Structures and Algorithms

- **Euclidean Distance:** $d_E(x, y) = \sqrt{\sum_{i=1}^p (x_i - y_i)^2}$, where x and y are p -dimensional vectors.
- **Hamming Distance:** Counts the number of differing bits between two binary vectors.
- **Matching Coefficient:** Hamming distance normalized by the number of attributes/bits.
- **Recursive Problem Solving:** Do part of the work and reduce the rest into a similar but smaller problem.
- **Structural Recursion:** Traversal of data structures like trees using recursion. Breadth-First Search visits nodes level by level; Depth-First Search traverses down a branch.

3.7 SAT Solvers

- **SAT:** Determining whether a given propositional formula is satisfiable.
- **3-SAT:** Reducing the unrestricted SAT problem to 3-SAT by transforming each clause $l_1 \vee \dots \vee l_n$ to a conjunction of $n - 2$ clauses. The resulting formula is equisatisfiable with the original, meaning they have the same set of satisfying assignments.
- **CNF Conversion:** A set of rules to transform logical expressions into Conjunctive Normal Form (CNF). The rules define how to convert various logical operators (implication, conjunction, disjunction, biconditional, negation) into CNF.
- **DPLL:** A recursive algorithm that improves upon the Naïve SAT algorithm by incorporating Boolean Constraint Propagation as a sub-algorithm within DPLL that repeatedly identifies and assigns values to literals in unit clauses (clauses with only one unassigned literal).
- **Resolution Rule:** An inference rule that derives a new clause from two clauses containing a literal and its negation. The new clause combines the remaining literals from the original clauses using logical OR.
- **Clause Learning:** A technique used to learn from conflicts encountered during the DPLL search. When a conflict occurs, a concise reason (a clause) is learned to prevent similar conflicts in the future, improving efficiency.

3.8 Recommender Systems

- **Modeling Systematic Biases in Recommender Systems:** $r_{ij} \approx \mu + b_i + b_j + u_i \cdot v_j$, where μ is the overall mean rating, b_i is the bias for user i , b_j is the bias for movie j , and $u_i \cdot v_j$ represents the user-movie interaction.

3.9 Fairness in Machine Learning

- **Fairness through Blindness:** Ignore all irrelevant/protected attributes. However, it is possible to predict these attributes with high accuracy from other data.
- **Group Fairness:** $P(\text{outcome} | S) = P(\text{outcome} | T)$, where S is a minority group and T is the rest of the population. This is not a strong enough notion of fairness.

- **Individual Fairness:** Treat similar individuals similarly. Similar individuals should be treated similarly for the purpose of the classification task and should be similarly distributed over outcomes. How to achieve this is an active area of research.

3.10 Feature Extraction

- **Bag of Features:** Represent images by frequencies of “visual words” (Bags of features)

3.11 Time Series Analysis

- **Moving Average:** Replace each pixel with an average of all the values in its neighborhood
- **Weighted Moving Average:** Add weights to our moving average

3.12 Robotics

- **Robotic Assembly:** Representing robotic assembly as a search problem. States are joint angles and part positions, actions are joint movements, the goal is complete assembly, and the path cost is assembly time.